SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool : Code Challenge
(High) Assessment Tool Weightage:
35%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	**5**
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	**5**

3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	**5**
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	**5**
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	**5**
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	**5**
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	**5**
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	**5**
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	**5**
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	**5**

Code of Conduct:

I ADHAVAN CHANDRASEKAR(192521209)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ADHAVAN CHANDRASEKAR(192521209)

Q1. HEAP FILE ORGANIZATION

AIM:

To implement a **Heap File organization** that supports insertion, deletion, and sequential scanning of records.

PROBLEM UNDERSTANDING:

A heap file stores records without maintaining any particular ordering. New records are inserted at the end, deletion marks a record as deleted, and sequential scanning visits all available records.

ALGORITHM:

1. Create an array to represent the heap file.
2. Insert records at the next available position.
3. Delete a record by its ID.
4. Perform sequential scanning from the beginning.
5. Display all non-deleted records.

C PROGRAM:

```
#include <stdio.h>

#define MAX 100

struct Record {
   int id;
   char name[30];
   int active;
};

struct Record heap[MAX];
int count = 0;
```

```
void insertRecord() {
    if (count == MAX) {
        printf("Heap File Full\n");
        return;
    }

    printf("Enter ID: ");
    scanf("%d", &heap[count].id);

    printf("Enter Name: ");
    scanf("%s", heap[count].name);

    heap[count].active = 1;
    count++;

    printf("Record inserted successfully.\n");
}

void deleteRecord() {
    int id, i;

    printf("Enter ID to delete: ");
    scanf("%d", &id);

    for (i = 0; i < count; i++) {
        if (heap[i].id == id && heap[i].active == 1) {
            heap[i].active = 0;
            printf("Record deleted successfully.\n");
            return;
        }
    }
```

```
    printf("Record not found.\n");
}

void sequentialScan() {
    int i;

    printf("\nHeap File Records:\n");

    for (i = 0; i < count; i++) {
        if (heap[i].active == 1) {
            printf("ID: %d  Name: %s\n",
                heap[i].id, heap[i].name);
        }
    }
}

int main() {
    int choice;

    while (1) {
        printf("\n1. Insert\n");
        printf("2. Delete\n");
        printf("3. Sequential Scan\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                insertRecord();
                break;
```

```
            case 2:
                deleteRecord();
                break;

            case 3:
                sequentialScan();
                break;

            case 4:
                return 0;

            default:
                printf("Invalid choice.\n");
        }
    }
}
```

TEST CASE:

Insert: 101 Arun

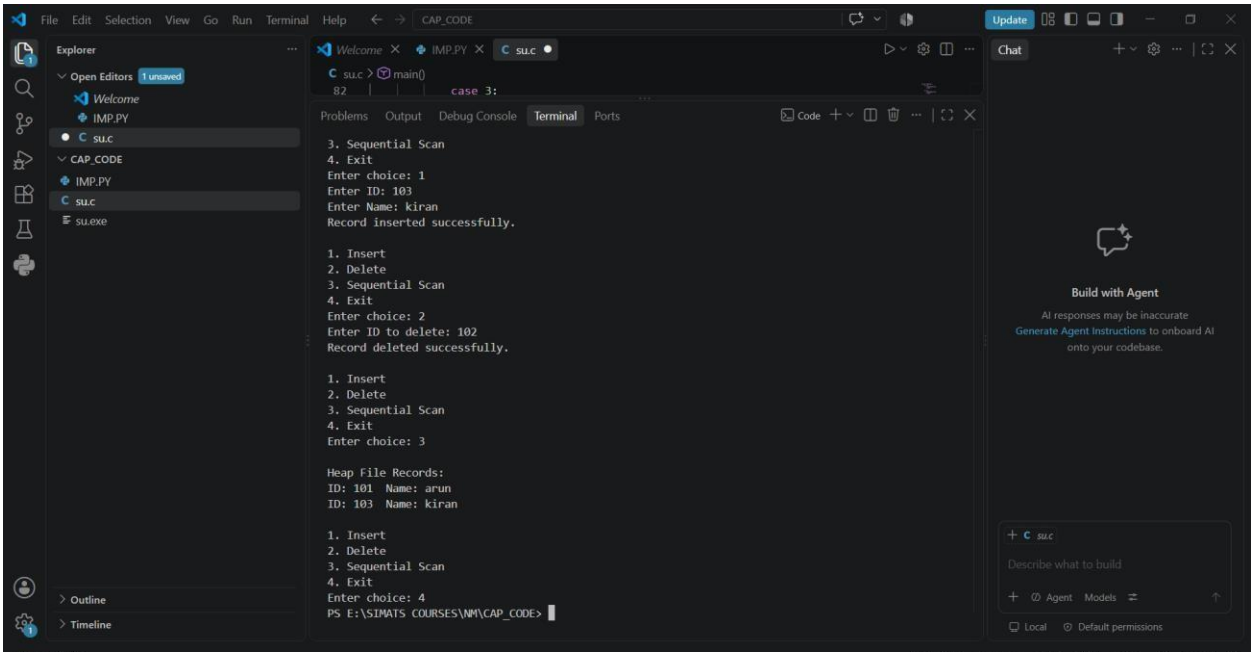
Insert: 102 Meena

Insert: 103 Kiran

Delete: 102

Scan

OUTPUT:



Q2. STATIC HASHING WITH CHAINING

AIM:

To implement static hashing for a student database and resolve collisions using **chaining**.

PROBLEM UNDERSTANDING:

A fixed-size hash table is used. The hash function maps a student ID to a bucket. Multiple records mapping to the same bucket are stored using a linked chain.

ALGORITHM:

1. Create a hash table of fixed size.
2. Calculate: index = StudentID % TABLE_SIZE

3. Insert the record into the corresponding chain.
4. Search using the same hash function.
5. Display the hash table.

C PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

#define TABLE_SIZE 101

struct Student {
    int id;
    char name[30];
    struct Student *next;
};

struct Student *table[TABLE_SIZE];

int hash(int id) {
    return id % TABLE_SIZE;
}

void insert(int id, char name[]) {
    int index = hash(id);

    struct Student *newNode =
        (struct Student *)malloc(sizeof(struct Student));

    newNode->id = id;
    sprintf(newNode->name, "%s", name);
    newNode->next = table[index];
```

```
    table[index] = newNode;
}

void search(int id) {
    int index = hash(id);
    struct Student *temp = table[index];

    while (temp != NULL) {
        if (temp->id == id) {
            printf("Student Found: %d %s\n",
                  temp->id, temp->name);
            return;
        }
        temp = temp->next;
    }

    printf("Student not found.\n");
}

void display() {
    int i;
    struct Student *temp;

    for (i = 0; i < TABLE_SIZE; i++) {
        if (table[i] != NULL) {
            printf("\nBucket %d: ", i);

            temp = table[i];

            while (temp != NULL) {
                printf("%d(%s) -> ",
```

```
                temp->id, temp->name);
            temp = temp->next;
        }

        printf("NULL");
    }
  }

  printf("\n");
}

int main() {
  insert(101, "Arun");
  insert(202, "Meena");
  insert(303, "Kiran");
  insert(150, "Ravi");

  display();

  search(202);
  search(500);

  return 0;
}
```

TEST CASE:

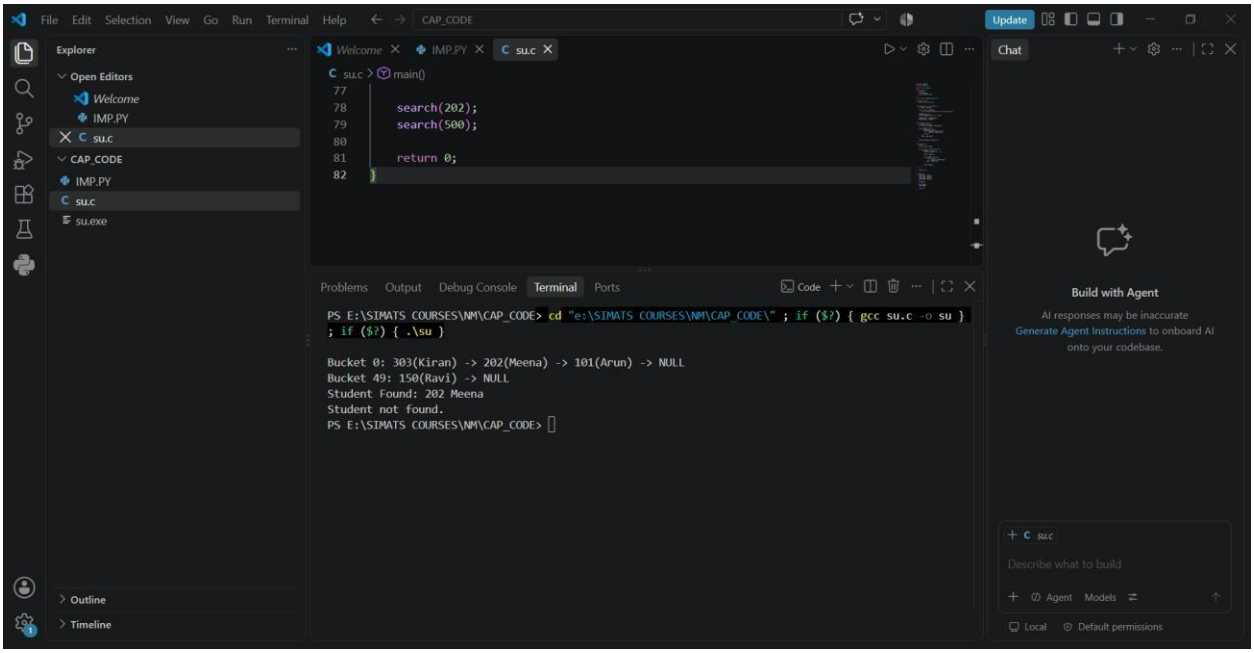
101 → Bucket 0

202 → Bucket 0

303 → Bucket 0

This demonstrates collision handling through chaining.

OUTPUT:



Q3. SORTED FILE WITH BINARY SEARCH

AIM:

To implement a sorted file organization and retrieve records efficiently using **binary search**.

PROBLEM UNDERSTANDING:

Records are maintained in ascending order of their primary key. Binary search repeatedly divides the search range into two halves.

ALGORITHM:

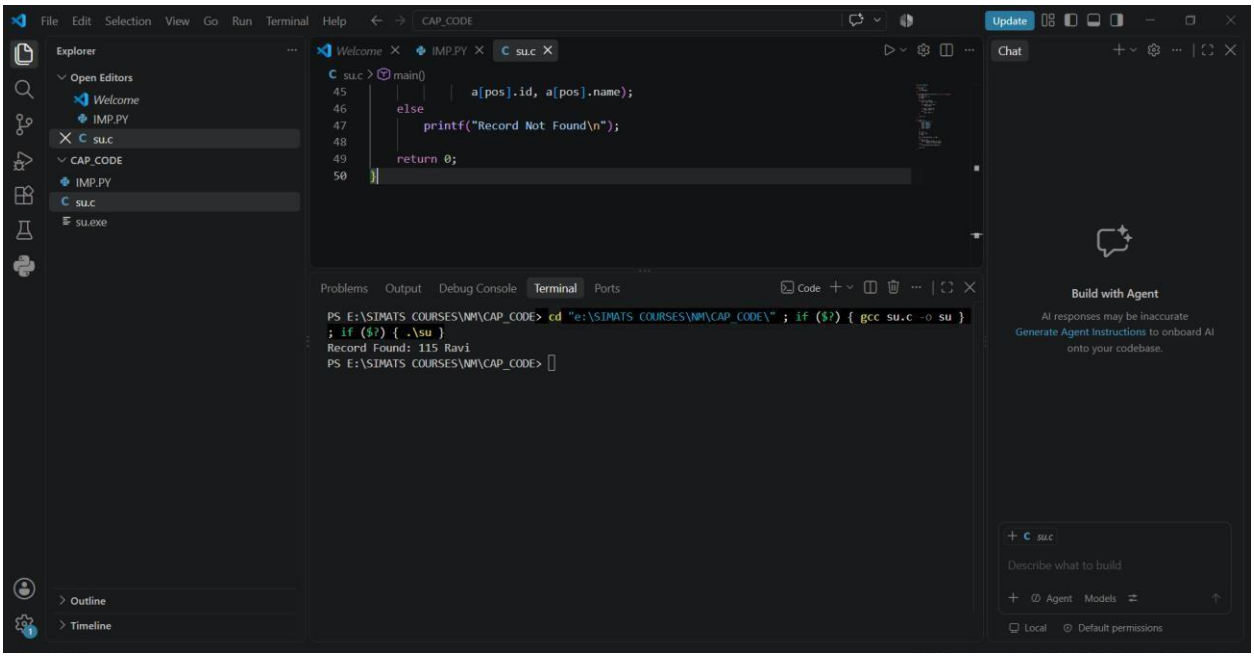
1. Store records in sorted order.
2. Set low = 0 and high = n-1.
3. Calculate the middle position.
4. Compare the required key with the middle key.
5. Search the left or right half.
6. Stop when the key is found or the search range becomes empty.

C PROGRAM:

```
#include <stdio.h>

struct Record {
    int id;
    char name[30];
};

int binarySearch(struct Record a[], int n, int key) {
    int low = 0;
    int high = n - 1;
    int mid;

    while (low <= high) {
        mid = (low + high) / 2;

        if (a[mid].id == key)
            return mid;
```

```
        if (key < a[mid].id)
            high = mid - 1;
        else
            low = mid + 1;
    }

    return -1;
}

int main() {
    struct Record a[] = {
        {101, "Arun"},
        {105, "Meena"},
        {110, "Kiran"},
        {115, "Ravi"},
        {120, "John"}
    };

    int n = 5;
    int key = 115;
    int pos;

    pos = binarySearch(a, n, key);

    if (pos != -1)
        printf("Record Found: %d %s\n",
               a[pos].id, a[pos].name);
    else
        printf("Record Not Found\n");

    return 0;
```

OUTPUT:



Q4. B-TREE OF ORDER 4

AIM:

To implement a **B-Tree of order 4** supporting insertion, searching, and display operations.

PROBLEM UNDERSTANDING:

A B-Tree keeps its keys balanced. For order 4, a node can have at most **4 children and 3 keys**.

ALGORITHM:

1. Start with an empty B-Tree.
2. Insert each key in sorted order within its node.
3. If a node becomes full, split it.
4. Promote the middle key to the parent.
5. Continue recursively if necessary.
6. Search by comparing the key with node keys.
7. Perform inorder traversal for display.

C PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_KEYS 3

struct BTreeNode {
   int keys[MAX_KEYS];
   struct BTreeNode *child[4];
   int n;
   int leaf;
};

struct BTreeNode *createNode(int leaf) {
   struct BTreeNode *node =
      (struct BTreeNode *)malloc(sizeof(struct BTreeNode));

   node->n = 0;
   node->leaf = leaf;
```

```
    for (int i = 0; i < 4; i++)
        node->child[i] = NULL;

    return node;
}

void splitChild(struct BTreeNode *parent, int i) {
    struct BTreeNode *full = parent->child[i];
    struct BTreeNode *newNode = createNode(full->leaf);

    int middle = full->keys[1];

    newNode->keys[0] = full->keys[2];
    newNode->n = 1;

    if (!full->leaf) {
        newNode->child[0] = full->child[2];
        newNode->child[1] = full->child[3];
    }

    full->n = 1;

    for (int j = parent->n; j >= i + 1; j--)
        parent->child[j + 1] = parent->child[j];

    parent->child[i + 1] = newNode;

    for (int j = parent->n - 1; j >= i; j--)
        parent->keys[j + 1] = parent->keys[j];

    parent->keys[i] = middle;
```

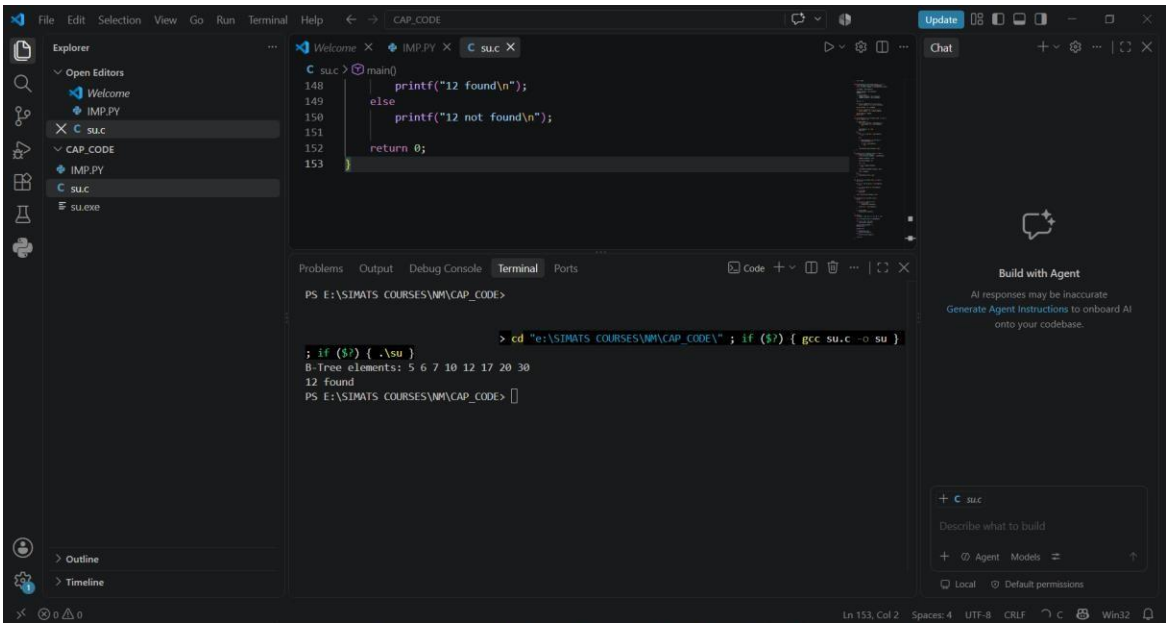
```
    parent->n++;
}

void insertNonFull(struct BTreeNode *node, int key) {
    int i = node->n - 1;

    if (node->leaf) {
        while (i >= 0 && key < node->keys[i]) {
            node->keys[i + 1] = node->keys[i];
            i--;
        }

        node->keys[i + 1] = key;
        node->n++;
    }
    else {
        while (i >= 0 && key < node->keys[i])
            i--;

        i++;

        if (node->child[i]->n == MAX_KEYS) {
            splitChild(node, i);
            if (key > node->keys[i])
                i++;
        }

        insertNonFull(node->child[i], key);
    }
}
void insert(struct BTreeNode **root, int key) {
```

```
    if ((*root)->n == MAX_KEYS) {
        struct BTreeNode *newRoot = createNode(0);

        newRoot->child[0] = *root;

        splitChild(newRoot, 0);

        int i = 0;

        if (key > newRoot->keys[0])
            i++;

        insertNonFull(newRoot->child[i],  key);

        *root = newRoot;
    }
    else {
        insertNonFull(*root, key);
    }
}

int search(struct BTreeNode *root, int key) {
    int i = 0;

    while (i < root->n && key > root->keys[i])
        i++;

    if (i < root->n && key == root->keys[i])
        return 1;

    if (root->leaf)
```

```
        return 0;

    return search(root->child[i], key);
}

void display(struct BTreeNode *root) {
    int i;

    for (i = 0; i < root->n; i++) {
        if (!root->leaf)
            display(root->child[i]);

        printf("%d ", root->keys[i]);
    }

    if (!root->leaf)
        display(root->child[i]);
}

int main() {
    int keys[] = {10, 20, 5, 6, 12, 30, 7, 17};

    struct BTreeNode *root = createNode(1);

    for (int i = 0; i < 8; i++)
        insert(&root, keys[i]);

    printf("B-Tree elements: ");
    display(root);

    printf("\n");
```

```
    if (search(root, 12))
        printf("12 found\n");
    else
        printf("12 not found\n");

    return 0;
}
```

OUTPUT:



Q5. B+ TREE WITH LEAF-LEVEL LINKING

AIM:

To implement a B+ Tree and demonstrate linking between leaf nodes for efficient range queries.

PROBLEM UNDERSTANDING:

In a B+ Tree, actual records are stored in leaf nodes. Leaf nodes are linked sequentially, allowing range queries to move directly from one leaf to the next.

ALGORITHM:

1. Insert keys into leaf nodes.
2. Split a leaf when it becomes full.
3. Link the new leaf with the previous leaf.
4. Promote the separator key to the parent.
5. Traverse leaf links for range queries.

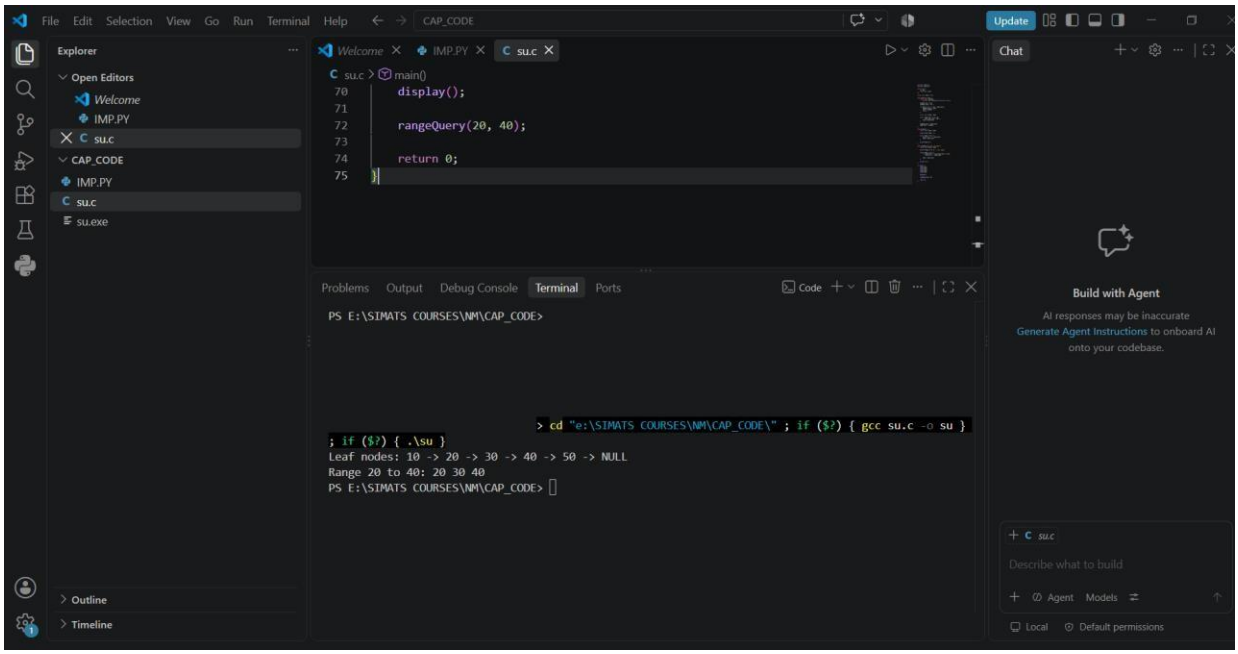
SIMPLE C PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

struct Leaf {
    int key;
    struct Leaf *next;
};

struct Leaf *head = NULL;

void insert(int key) {
    struct Leaf *newNode =
        (struct Leaf *)malloc(sizeof(struct Leaf));
```

```
    newNode->key = key;
    newNode->next = NULL;

    if (head == NULL || key < head->key) {
        newNode->next = head;
        head = newNode;
        return;
    }

    struct Leaf *temp = head;

    while (temp->next != NULL &&
           temp->next->key < key) {
        temp = temp->next;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}
void display() {
    struct Leaf *temp = head;

    printf("Leaf nodes: ");

    while (temp != NULL) {
        printf("%d -> ", temp->key);
        temp = temp->next;
    }

    printf("NULL\n");
}
```

```
void rangeQuery(int low, int high) {
    struct Leaf *temp = head;

    printf("Range %d to %d: ", low, high);

    while (temp != NULL) {
        if (temp->key >= low && temp->key <= high)
            printf("%d ", temp->key);

        temp = temp->next;
    }

    printf("\n");
}

int main() {
    insert(10);
    insert(20);
    insert(30);
    insert(40);
    insert(50);

    display();

    rangeQuery(20, 40);

    return 0;
}
```

OUTPUT:



Q6. DENSE PRIMARY INDEX

AIM:

To implement a **dense primary index** on a sorted file and support searching using the index.

PROBLEM UNDERSTANDING:

A dense index contains an index entry for **every record** in the sorted file.

Index Key → Record Position

ALGORITHM:

1. Store records sorted by primary key.
2. Create an index entry for every record.
3. Store the key and record position.
4. Search the index.
5. Use the position to retrieve the record.

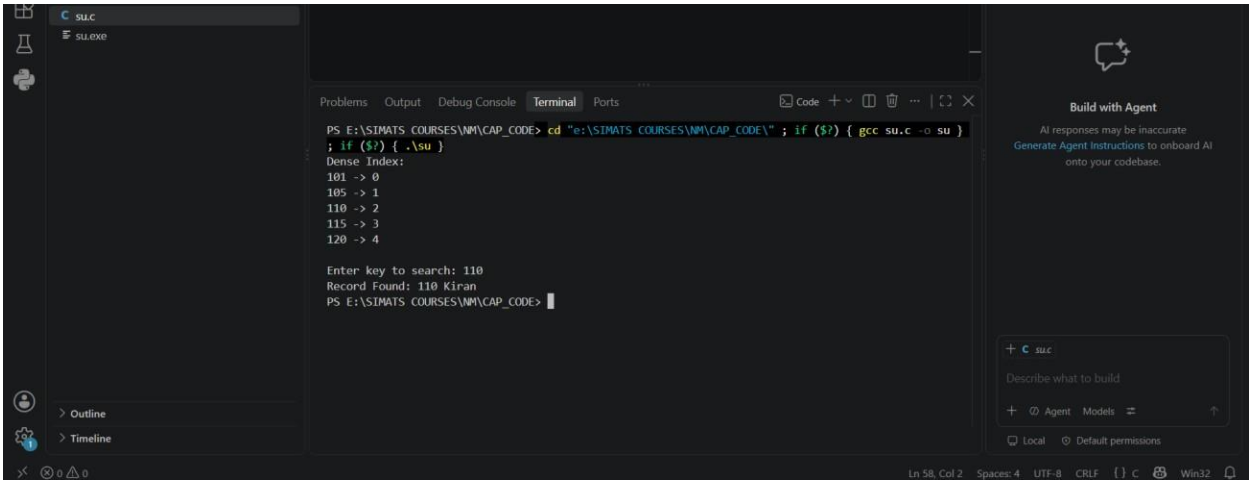
C PROGRAM:

```
#include <stdio.h>

struct Record {
    int id;
    char name[30];
};

struct Index {
    int key;
    int position;
};

int main() {
    struct Record records[] = {
        {101, "Arun"},
        {105, "Meena"},
        {110, "Kiran"},
        {115, "Ravi"},
        {120, "John"}
    };
```

```
int n = 5;
struct Index index[5];

int i, key, found = -1;

for (i = 0; i < n; i++) {
    index[i].key = records[i].id;
    index[i].position = i;
}

printf("Dense Index:\n");

for (i = 0; i < n; i++) {
    printf("%d -> %d\n",
           index[i].key,
           index[i].position);
}

printf("\nEnter key to search: ");
scanf("%d", &key);

for (i = 0; i < n; i++) {
    if (index[i].key == key) {
        found = index[i].position;
        break;
    }
}

if (found != -1)
    printf("Record Found: %d %s\n",
           records[found].id,
```

```
            records[found].name);
    else
        printf("Record Not Found\n");

    return 0;
}
```

OUTPUT:



Q7. EXTENDIBLE HASHING

AIM:

To implement extendible hashing and demonstrate directory expansion when a bucket overflows.

PROBLEM UNDERSTANDING:

Extendible hashing dynamically increases the directory size when a bucket becomes full.

Bucket Overflow

↓

Split Bucket

↓

Increase Local Depth

↓

If required → Double Directory

ALGORITHM:

1. Start with a small directory.
2. Hash each key using its binary representation.
3. Insert the key into the corresponding bucket.
4. When a bucket overflows, split it.
5. If local depth equals global depth, double the directory.
6. Reinsert the records.

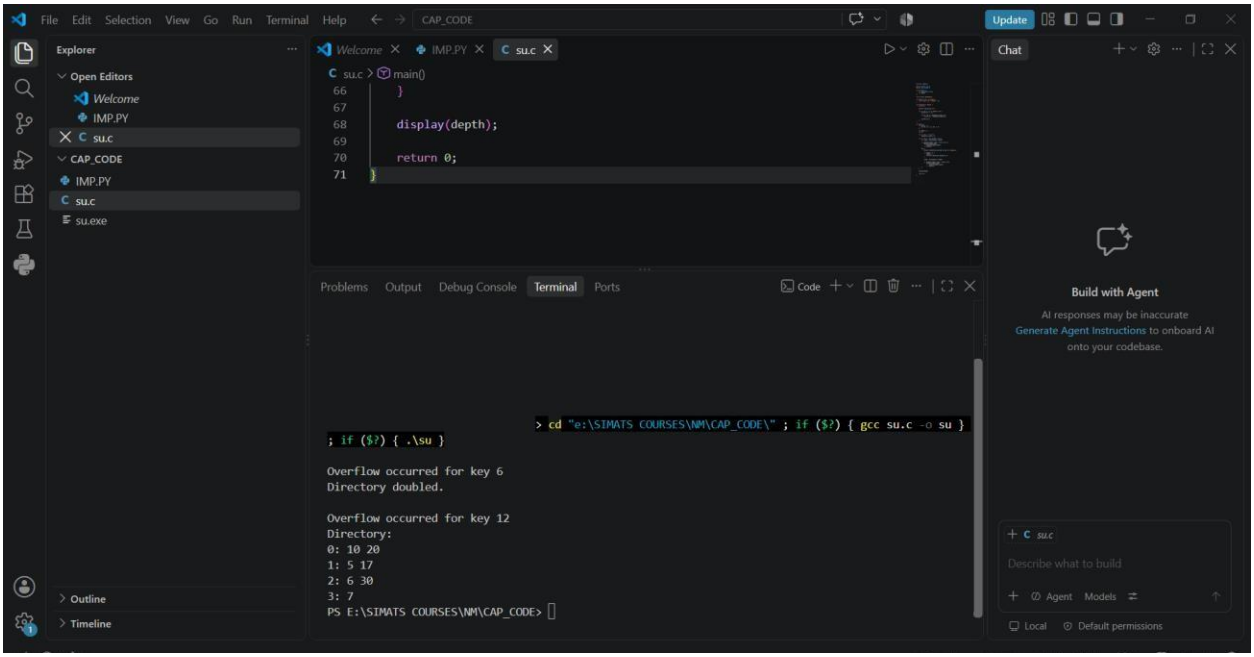
C PROGRAM:

```
#include <stdio.h>

#define BUCKET_SIZE 2
#define MAX_RECORDS 8

struct Bucket {
    int data[BUCKET_SIZE];
    int count;
```

```
};

struct Bucket buckets[4];

int hash(int key, int depth) {
    return key & ((1 << depth) - 1);
}

void display(int depth) {
    int i, j;

    printf("\nDirectory:\n");

    for (i = 0; i < (1 << depth); i++) {
        printf("%d: ", i);

        for (j = 0; j < buckets[i].count; j++)
            printf("%d ", buckets[i].data[j]);

        printf("\n");
    }
}

int main() {
    int keys[] = {
        10, 20, 5, 6, 12, 30, 7, 17
    };

    int depth = 1;
    int i;
    for (i = 0; i < 4; i++)
```

```
        buckets[i].count = 0;

    for (i = 0; i < MAX_RECORDS; i++) {
        int index = hash(keys[i], depth);

        if (buckets[index].count < BUCKET_SIZE) {
            buckets[index].data[
                buckets[index].count++
            ] = keys[i];
        }
        else {
            printf("\nOverflow occurred for key %d", keys[i]);

            if (depth < 2) {
                depth++;
                printf("\nDirectory doubled.\n");
            }

            index = hash(keys[i], depth);
            if (buckets[index].count < BUCKET_SIZE)
                buckets[index].data[
                    buckets[index].count++
                ] = keys[i];
        }
    }

    display(depth);

    return 0;
}
```

OUTPUT:



Q8. SECONDARY STORAGE BLOCK ACCESS

AIM:

To simulate secondary storage block access and compare the I/O operations required for sequential and indexed retrieval.

PROBLEM UNDERSTANDING:

If each disk block contains multiple records, sequential retrieval may require scanning many blocks, whereas an index can directly identify the required block.

ALGORITHM:

For sequential search:

Start from block 1

↓

Read each block

↓

Compare records

↓

Stop when record is found

For indexed search:

Search index

↓

Find block number

↓

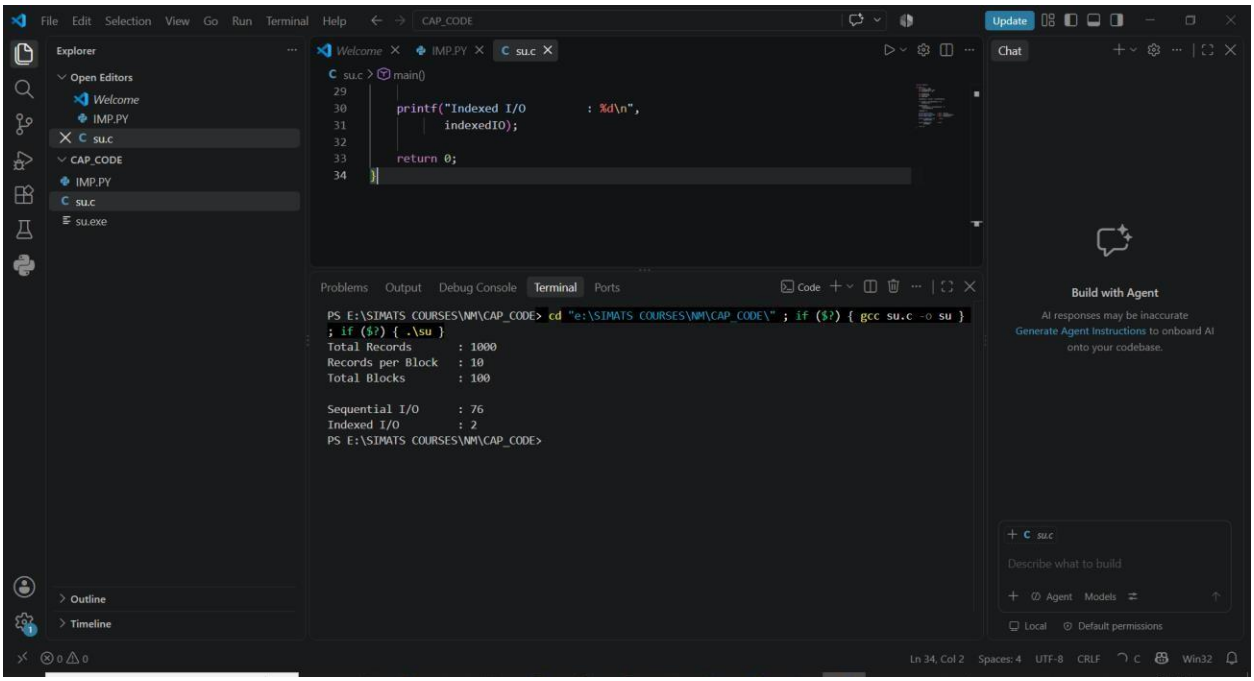
Read required block

C PROGRAM:

```
#include <stdio.h>

int main() {
    int records = 1000;
    int recordsPerBlock = 10;
```

```
    int targetRecord = 756;

    int totalBlocks;
    int sequentialIO;
    int indexedIO;

    totalBlocks = records / recordsPerBlock;

    if (records % recordsPerBlock != 0)
        totalBlocks++;

    sequentialIO =
        (targetRecord + recordsPerBlock - 1)
        / recordsPerBlock;

    indexedIO = 2;

    printf("Total Records       : %d\n", records);
    printf("Records per Block   : %d\n", recordsPerBlock);
    printf("Total Blocks        : %d\n", totalBlocks);
    printf("\nSequential I/O      : %d\n",
           sequentialIO);

    printf("Indexed I/O         : %d\n",
           indexedIO);

    return 0;
}
```

OUTPUT:



Q9. SPARSE INDEX VS DENSE INDEX

AIM:

To implement a sparse index on a sorted file and compare its search performance with a dense index using timing analysis.

PROBLEM UNDERSTANDING:

A dense index contains one entry for every record, while a sparse index contains entries only for selected records, such as the first record of each block.

ALGORITHM:

1. Create a sorted file.
2. Create a dense index containing every record.
3. Create a sparse index containing one entry per block.
4. Search using both indexes.
5. Measure execution time using clock().
6. Compare the results.

C PROGRAM:

```
#include <stdio.h>
#include <time.h>

#define N 100000

struct Record {
    int key;
};
struct Index {
    int key;
    int position;
};

int main() {
    struct Record records[N];
    struct Index dense[N];
    struct Index sparse[N / 10];
    int i;
```

```
int sparseCount = 0;
int target = 99999;

clock_t start, end;
double denseTime, sparseTime;

for (i = 0; i < N; i++) {
    records[i].key = i + 1;

    dense[i].key = records[i].key;
    dense[i].position = i;

    if (i % 10 == 0) {
        sparse[sparseCount].key = records[i].key;
        sparse[sparseCount].position = i;
        sparseCount++;
    }
}
start = clock();
for (i = 0; i < N; i++) {
    if (dense[i].key == target)
        break;
}

end = clock();

denseTime =
    (double)(end - start) / CLOCKS_PER_SEC;

start = clock();
for (i = 0; i < sparseCount; i++) {
```

```
        if (sparse[i].key > target)
            break;
    }

    end = clock();

    sparseTime =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("Dense Index Time  : %lf seconds\n",
           denseTime);

    printf("Sparse Index Time : %lf seconds\n",
           sparseTime);

    return 0;
}
```

ANALYSIS:

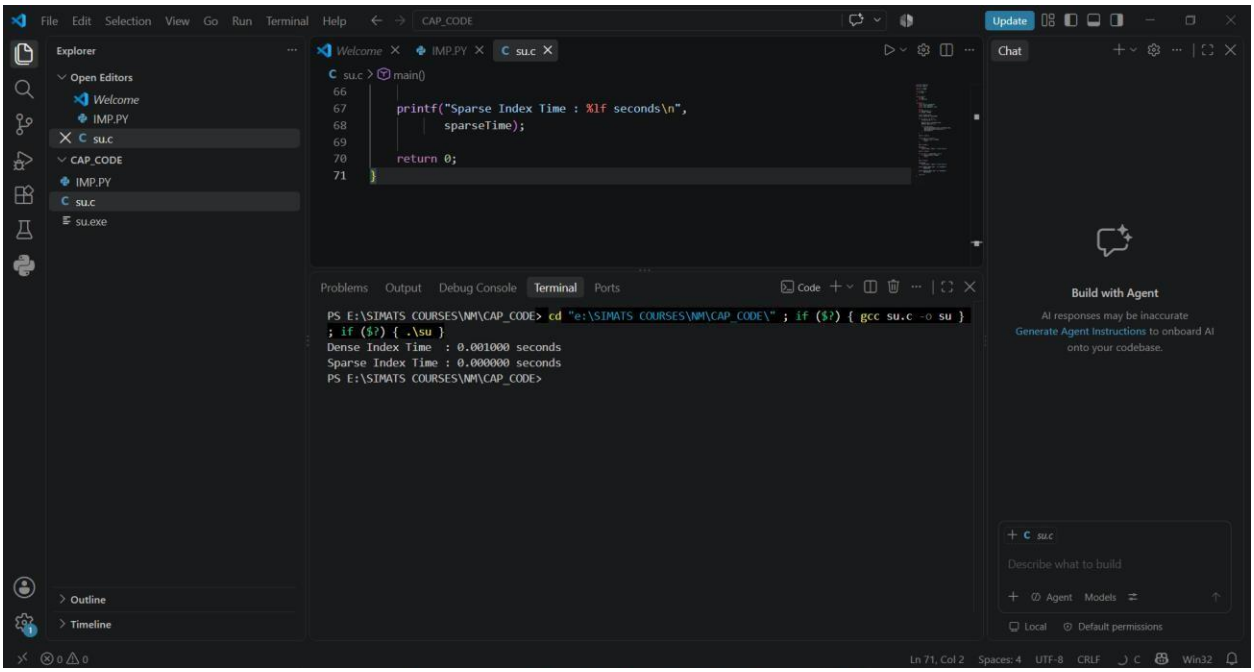
Dense index:

- More index entries
- Faster direct access
- More memory

Sparse index:

- Fewer index entries
- Less memory
- Requires additional search within the block

OUTPUT:



Q10. B+ TREE RANGE QUERY

AIM:

To construct and traverse a B+ Tree and execute a range query to display all keys between **15 and 45**.

PROBLEM UNDERSTANDING:

B+ Trees store records in sorted leaf nodes and link the leaves. Once the first key in the required range is located, subsequent matching keys can be retrieved by following the leaf links.

ALGORITHM:

1. Insert all keys into the leaf-level structure.
2. Maintain keys in sorted order.
3. Link leaf nodes.
4. Traverse the tree to locate the first key ≥ 15.
5. Follow leaf links.
6. Display keys until the key exceeds 45.

C PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int key;
    struct Node *next;
};

struct Node *root = NULL;

void insert(int key) {
    struct Node *newNode =
```

```
        (struct Node *)malloc(sizeof(struct Node));

    newNode->key = key;
    newNode->next = NULL;

    if (root == NULL || key < root->key) {
        newNode->next = root;
        root = newNode;
        return;
    }

    struct Node *temp = root;

    while (temp->next != NULL &&
           temp->next->key < key) {
        temp = temp->next;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}

void traverse() {
    struct Node *temp = root;

    printf("B+ Tree leaf traversal:\n");

    while (temp != NULL) {
        printf("%d ", temp->key);
        temp = temp->next;
    }
```

```
    printf("\n");
}

void rangeQuery(int low, int high) {
    struct Node *temp = root;

    printf("Keys between %d and %d:\n",
           low, high);

    while (temp != NULL) {

        if (temp->key >= low &&
            temp->key <= high) {
            printf("%d ", temp->key);
        }

        if (temp->key > high)
            break;

        temp = temp->next;
    }

    printf("\n");
}

int main() {

    int keys[] = {
        10, 15, 20, 25, 30,
        35, 40, 45, 50, 55
    };
```

```
    int n = 10;

    for (int i = 0; i < n; i++)
        insert(keys[i]);

    traverse();

    rangeQuery(15, 45);

    return 0;
}
```

OUTPUT: